

Testing PCA Algorithm

May 13, 2026

0.1 Testing the PCA Algorithm written from scratch using Numpy

```
[1]: # Importing Libraries
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
```

```
[2]: # Importing PCA Algorithm
import pca
```

```
[3]: # Loading iris dataset from sklearn
from sklearn import datasets
data = datasets.load_iris()['data']
data.shape
```

```
[3]: (150, 4)
```

```
[4]: # Creating Object called model from PCA Class
model = pca.PCA()
```

```
[5]: # Fitting the iris dataset
# It will center the dataset and calculate covariance matrix , eigen vales and
    ↪ eigen vectors
model.fit_data(data)
```

Dataset fitted successfully

```
[6]: # Projecting the centered dataset onto lower dimension and storing them
# Default will be Projecting centered dataset onto top k principal components
    ↪ which explains ~95% of data
data_lowdim = model.compress()
```

The data projected into the low dimensional PCA space of 2 dimension

```
[7]: # Reconstructing dataset using that top k principal components
reconstructed_dataset = model.represent()
```

```
[8]: # Calculating Squared error
error = np.sum((data - reconstructed_dataset)**2)
```

```
error
```

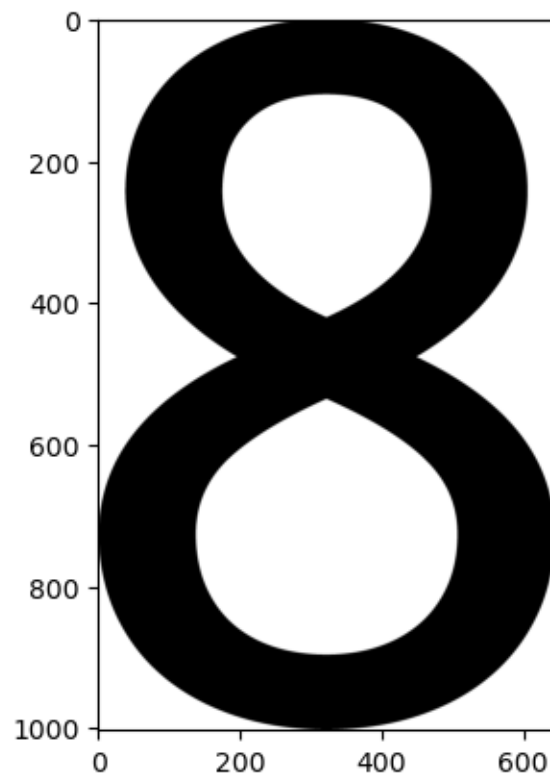
```
[8]: 15.204644359438953
```

0.1.1 Running PCA on Image

```
[9]: #Importing Image  
img = Image.open('eight.png')  
  
# Converting to NumPy array  
img_array = np.asarray(img)  
img_array.shape
```

```
[9]: (1003, 644, 4)
```

```
[10]: # Plotting the Image using Image array  
plt.imshow(img_array);
```



```
[11]: # Reshaping the Image Array  
X = img_array.reshape(1003,644*4)  
X.shape
```

```
[11]: (1003, 2576)
```

```
[12]: model.fit_data(X)
```

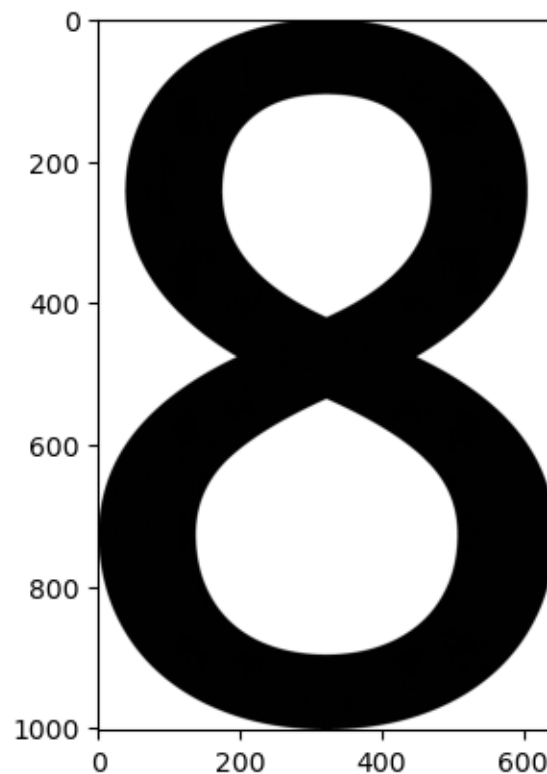
Dataset fitted successfully

```
[13]: #Projecting centered dataset onto top k principal components which explains  
      ↪100% of data  
      img_lowdim = model.compress(100)
```

The data projected into the low dimensional PCA space of 2576 dimension

```
[14]: # Reconstructing the dataset using all of the Principal Components  
      reconstructed_image = model.represent()
```

```
[15]: # Reshaping the Image array to its original shape  
      reshaped_image = reconstructed_image.reshape(1003, 644, 4)  
      #Forcing the element values to stay between 0 and 255  
      reconstructed_image_final = np.clip(reshaped_image, 0, 255).astype('uint8')  
      plt.imshow(reconstructed_image_final);
```



```
[16]: # Calculating Squared error  
error = np.sum((X - reconstructed_image)**2)  
error
```

```
[16]: 3.7346406101836365e-20
```

```
[17]: # In numpy the error won't be zero (~0)  
# But we can use allclose() which returns True if two arrays are element-wise  
↪ equal within a specified tolerance  
print(np.allclose(X,reconstructed_image))
```

```
True
```